

Bit Operations

CSC 230 : C and Software Tools

NC State Department of Computer
Science

Topics for Today

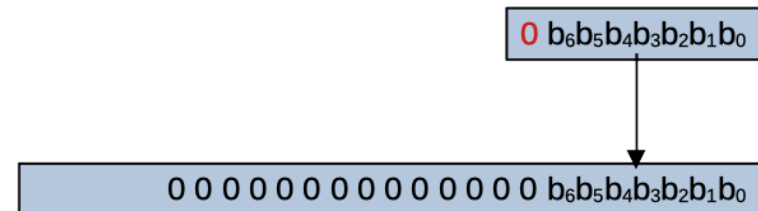
- Bitwise operator review
- Bitwise tasks
 - Setting / clearing / testing selected bits
 - Copying selected bits
 - Movable masks
 - Manipulating a bit field
- Bit fields in structs

Looking Inside a Value

- Internally, int and double values are represented with bit patterns
- A good programming language lets us ignore details of how this is done
 - We can think of an int as ... well, an integer
- But, sometimes we want to work with individual bits in a value ... why?
 - Maybe we could pack more values into limited memory.
 - Maybe we need to talk to hardware or work with a file format that requires particular bit patterns
 - Maybe we need to perform compression or encryption
 - Maybe we can better exploit bit-level parallelism in the hardware

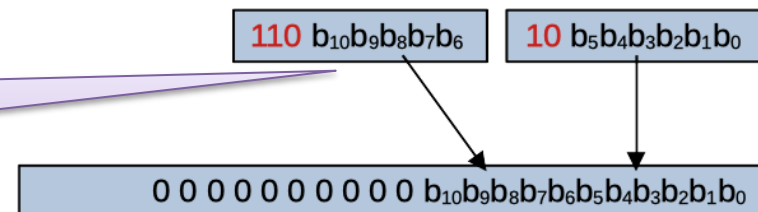
Application : UTF-8 Encoding

- UTF-8 is a file format for non-ASCII text.
 - A character code can be up to 21 bits.
 - It uses just some bits from each byte.
- A character with a 7-bit code uses just one byte.



- An 11-bit code is spread across two bytes.

We need to be able to extract these bits and put them together.



Working with Bits

- C99 provides **no standard way** to ...

- ... give a literal constant in binary

```
i = 011001011;
```

- ... read an integer as a string of binary digits

```
scanf( "%b", &i );
```

- ... print a value as a string of binary digits

```
printf( "%b", i );
```

- What can we do?

- Use octal or hexadecimal

- Write our own code to work at the bit level.

Hexadecimal Review

- Each hexadecimal digit is like shorthand for four bits.

Hex Digit	Bit pattern
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

Hex Digit	Bit pattern
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

C Bit-Level Operators

- We have some operators just for manipulating values at the bit level

Operator	Description
++ --	Postincrement, postdecrement
++ --	Preincrement, predecrement
+ -	Unary positive, negative
!	Logical not
~	Bitwise complement
(type)	Type cast
sizeof	Well, it's the sizeof operator
* / %	Multiply, divide, mod
+ -	Add, subtract
<< >>	Bitwise left and right shift
< <= > >=	Relational operators

More C Operators

Operator	Description
== !=	Equals, not equals
&	Bitwise and
^	Bitwise exclusive or
	Bitwise (inclusive) or
&&	Logical and
	Logical or
? :	Ternary operator
= += -= *= ...	Assignment, and its friends
,	Comma operator

Looking at Bits

- Signed-ness matters
 - The same pattern of bits can look different as a signed or an unsigned value.

```
unsigned char x;  
signed char y;  
  
x = 0xA9; // binary 10101001 (that means 169)  
y = 0xA9; // binary 10101001 (here, that's -87)  
  
printf( "%d\n", x );  
printf( "%d\n", y );
```

These get converted to int when passed to printf.

Looking at Bits

- Signed-ness matters
 - The same pattern of bits can look different as a signed or an unsigned value.

```
unsigned char x;  
signed char y;  
  
x = 0xA9; // binary 10101001 (that means 169)  
y = 0xA9; // binary 10101001 (here, that's -87)  
  
printf( "%d\n", x );  
printf( "%d\n", y );
```

This will print 169

This will print -87

Looking at Bits

We talked
about this in
Lecture 8

- Width matters
 - Signed values will be sign-extended when converted to a wider type.
 - Remember, C promotes to int / unsigned int for any arithmetic operation.

```
unsigned char x;  
signed char y;  
  
x = 0xA9; // binary 10101001 (A9)  
y = 0xA9; // binary 10101001 (A9)  
  
printf( "%X\n", x );  
printf( "%X\n", y );
```

Prints A9
(like you'd expect)

Prints FFFFFFFA9
(that's sign extension)

Meet ~ (a Unary Bitwise Operator)

- Unary operator ~ is a **bitwise complement**

```
unsigned char x, y;  
  
x = 0x1A;    // binary 00011010  
y = ~x;      // binary 11100101  
  
printf( "%X\n", y );  
printf( "%X\n", ~x );
```

This will print E5, like you expect.

This will print FFFFFFFE5
x is expanded to an int before the complement is performed.

Friends of ~

- We have three different “not” operators.
- They each do slightly different job.

```
signed char x, y, z, w;
```

```
x = 0x1A; // binary 00011010
```

```
y = ~x; // binary 11100101
```

```
z = !x; // binary 00000000
```

```
w = -x; // binary 11100110
```

I flip all the bits.

I swap true $\leftrightarrow$ false

I perform the two's complement operation.

Binary Bitwise Operators

- The next three operators are binary, they take two operands
 - **&** is a bitwise **and**
a 1 in the result only if both corresponding input bits 1
 - **^** is bitwise **exclusive or**
a 1 in the result only if both corresponding input bits are different
 - **|** is bitwise **(inclusive) or**
a 1 in the result if either of the corresponding input bits are 1

Bitwise And

- Here's how bitwise and (&) works:



Zero and zero gives you zero.

Written as a single (infix) ampersand.

Bitwise And

- Here's how & works.



```
  1 0
& 1 0
-----
  1 0
```

I'm showing this like an elementary school math problem.

I can combine multiple bits at the same time.

One and one equals one.

Bitwise And

- Here's how & works.

```
  1 1 0 0 1 0 1 0
& 1 0 1 0 0 1 1 0
-----
  1 0 0 0 0 0 1 0
```

I can compute a result for all corresponding bits at once.

As usual for and, the result bit is 1 only if corresponding input bits are both 1.

- Logical and is different; it interprets each input value as a single truth value.

```
  1 1 0 0 1 0 1 0
&& 1 0 1 0 0 1 1 0
-----
  0 0 0 0 0 0 0 1
```

As a boolean, I'm true.

Me too.

So, the result is true.

Bitwise Or

- Bitwise Or is similar
- It computes each bit of the result based on corresponding bits of the operands.

```
  1 1 0 0 1 0 1 0
& 1 0 1 0 0 1 1 0
-----
  1 0 0 0 0 0 1 0
```

Written as a single
vertical bar.

```
  1 1 0 0 1 0 1 0
| 1 0 1 0 0 1 1 0
-----
  1 1 1 0 1 1 1 0
```

Resulting bit is 1 if either
input bit is 1.

Bitwise Or

- Bitwise Or is similar
- It computes each bit of the result based on corresponding bits of the operands.

```
  1 1 0 0 1 0 1 0
& 1 0 1 0 0 1 1 0
-----
  1 0 0 0 0 0 1 0
```

```
  1 1 0 0 1 0 1 0
| 1 0 1 0 0 1 1 0
-----
  1 1 1 0 1 1 1 0
```

Result bit is 0 only where both input bits are 0.

- Logical or is like this, but it considers each input to be one big truth value.

```
  1 1 0 0 1 0 1 0
|| 1 0 1 0 0 1 1 0
-----
  0 0 0 0 0 0 0 1
```

As a boolean, I'm true.

Me too.

So, the result is true.

Bitwise vs Logical Operators

- Bitwise And and Or don't care about short circuiting.

```
D = A & ( B | C );
```

I still get evaluated even if A is zero.

- Corresponding logical operators are guaranteed to short circuit.

```
D = A && ( B || C );
```

I won't get evaluated unless A is true (non-zero).

Exclusive Or

- One more bitwise operator, exclusive Or.

```
  1 1 0 0 1 0 1 0
& 1 0 1 0 0 1 1 0
-----
  1 0 0 0 0 0 1 0
```

```
  1 1 0 0 1 0 1 0
| 1 0 1 0 0 1 1 0
-----
  1 1 1 0 1 1 1 0
```

```
  1 1 0 0 1 0 1 0
^ 1 0 1 0 0 1 1 0
-----
  0 1 1 0 1 1 0 0
```

Written as a caret.

Result bit is 1 if one corresponding bit is 1.

Exclusive Or

- One more bitwise operator, exclusive Or.

```
  1 1 0 0 1 0 1 0
& 1 0 1 0 0 1 1 0
-----
  1 0 0 0 0 0 1 0
```

```
  1 1 0 0 1 0 1 0
| 1 0 1 0 0 1 1 0
-----
  1 1 1 0 1 1 1 0
```

```
  1 1 0 0 1 0 1 0
^ 1 0 1 0 0 1 1 0
-----
  0 1 1 0 1 1 0 0
```

Result bit is 0 if neither ...

... or both input bits are
1.

Exclusive Or

- One more bitwise operator, exclusive Or.

```
  1 1 0 0 1 0 1 0
& 1 0 1 0 0 1 1 0
-----
  1 0 0 0 0 0 1 0
```

```
  1 1 0 0 1 0 1 0
| 1 0 1 0 0 1 1 0
-----
  1 1 1 0 1 1 1 0
```

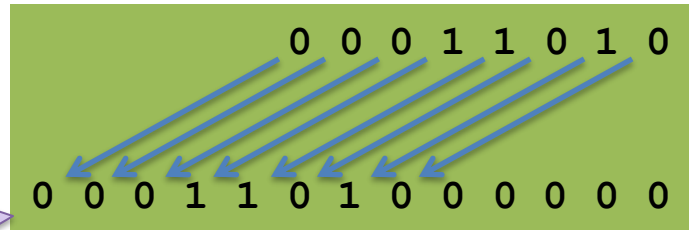
```
  1 1 0 0 1 0 1 0
^ 1 0 1 0 0 1 1 0
-----
  0 1 1 0 1 1 0 0
```

- Does this one have an equivalent logical operator? ... sort of.

Shift Operators

- Left logical shift of x by y bits: $x \ll y$
 - For example: $0x1A \ll 5$

If you shift far enough, you can lose bits off the high end.



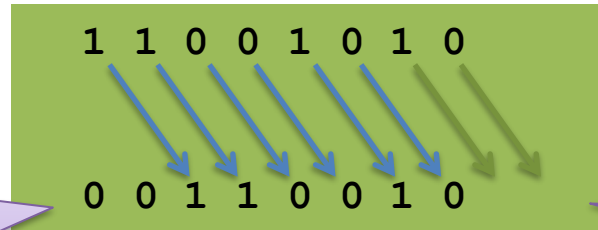
New zeros shifted in on this end.

- Operands should be integer types
- If x has n bits, y should be between 0 and $n-1$
- You can't left shift by a negative amount, but ...

Shift Operators

- **Right logical** shift of x by y bits: $x \gg y$
 - For example: $0xCA \gg 2$

You get new high-order zeros shifted in on the left.



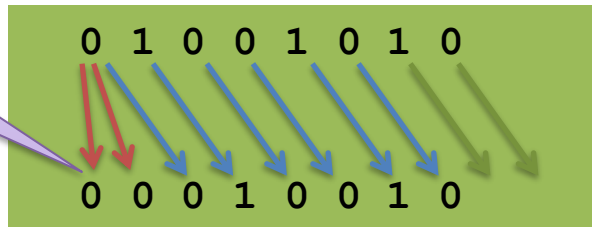
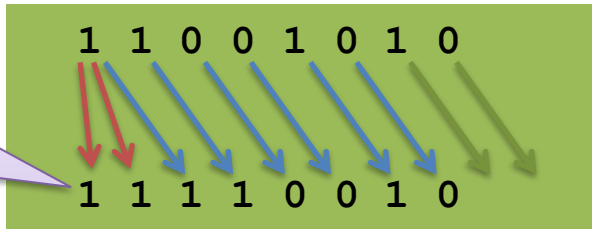
bits shifted past the low-order bit are discarded.

- Again, operands should be integer types
- If x has n bits, y should be between 0 and $n-1$
- Really, what gets shifted in on the left depends

Shift Operators

- Right **arithmetic** shift of x by y bits: $x \gg y$
 - Wait! This is the same operator.
 - Behavior depends on whether the left operand is **signed**.

For signed operands, you get copies of the high-order bit shifted in.



Shift Operators

- Why arithmetic shift?
 - For signed values, this maintains the sign.
- We can use left shift to multiply by 2 or any power of 2
 - $x \ll y$ is the same as $x * 2^y$
- And, we can use right shift like a divide operation
 - $x \gg y$ is the same as $x / 2^y$

```
unsigned int x y;
```

```
x = 75 << 2; // that's 300
```

```
y = 75 >> 2; // that's 18
```

But, this only works for positive numbers.

OK, How Do We Use These Operators?

- What can we do with these operators?
- Lots of useful things:
 - Clear selected bits to zero or set them to 1
 - Test if selected bits are zero or 1
 - Loop through individual bits.
 - Copy selected bits from x to y or to another position in x
 - Access a bit field as if it was a tiny integer.

Clearing Selected Bits

- We can think of the & operator as applying a *mask*

	0	1	1	0	1	1	0	0
&	1	1	0	0	1	0	1	0

	0	1	0	0	1	0	0	0

I'm the value you want to work with.

I'm the mask, deciding what bits to keep and discard.

- Wherever we have a 0 in the mask, a bit is cleared
- Wherever we have a 1, the original value passes through

Clearing Selected Bits

- So, we could clear just the low-order 4 bits

```
unsigned char x;  
x = 0xD3;           // binary 11010011  
x = x & 0xF0;       // mask  11110000  
                    // now    11010000
```

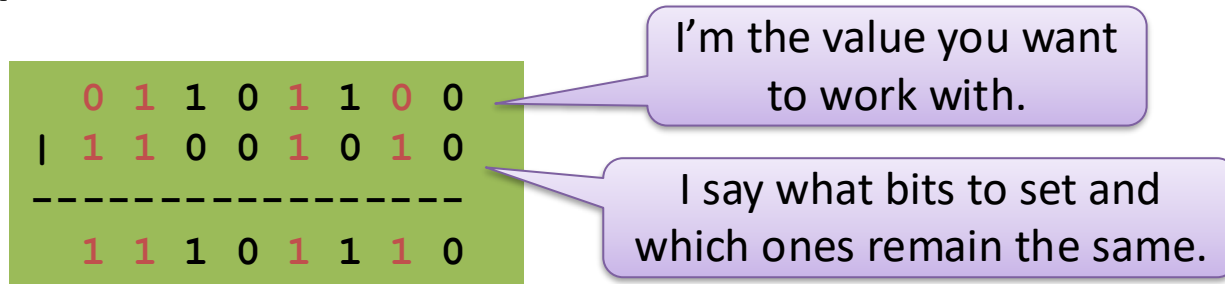
- Or, we could clear every other bit.

```
x = 0xD3;           // binary 11010011  
x &= 0x55;          // mask  01010101  
                    // now    01010001
```

We also have these for bitwise operators.

Setting Selected Bits

- Again, it's just how you think about the bitwise operators
- We can think of `|` as applying a different kind of mask



- Wherever we have a 1, a bit is set (made 1)
- Wherever we have a 0, the original value passes through

Setting Selected Bits

- So, we could set just the low-order 4 bits

```
unsigned char x;  
x = 0xD3;           // binary 11010011  
x = x | 0x0F;       // mask   00001111  
                    // now    11011111
```

- Or, we could set just the first and last.

```
x = 0xD6;           // binary 11010110  
x |= 0x81;          // mask   10000001  
                    // now    11010111
```

Inverting Selected Bits

- With the ^ operator, a 1 in the mask says where to flip a bit

	0	1	1	0	1	1	0	0
^	1	1	0	0	1	0	1	0

	1	0	1	0	0	1	1	0

I'm the value you want to work with.

I say what bits to change.

- We could use this to flip just the low-order bit

```
unsigned char x;  
x = 0xD3;           // binary 11010011  
x ^= 0x01;          // mask   00000001  
                     // now    11010010
```

Testing Selected Bits

- First, use & to mask off the bits you don't want.

0	1	1	0	1	1	0	0
&	0	0	0	0	1	1	1

0	0	0	0	1	1	0	0

- Then what?
 - If the result is zero, none of those bits were 1
 - If the result matches the mask, they were all 1

```
unsigned char x, mask;  
x = 0x6C;           // binary 01101100  
mask = 0x0F;        // mask    00001111  
if ( ( x & mask ) != 0 ) //      00001100 != 00000000  
    ...;  
if ( ( x & mask ) == mask ) // result 00001100 == 00001111  
    ...;
```

Careful here,
precedence.

Counting or Printing Bits

- So, if we can build a mask, we can get to any bits we want.
- What if we want to be able to access every bit ... individually.
 - Say, we want to iterate over the bits in an int.
- We can build a **movable mask** for this
 - $0x01 \ll i$ is a mask with a 1 just in bit position i
 - And, of course, $\sim (0x01 \ll i)$ is the complementary mask

Is this the same as
 $\sim 0x01 \ll i$?

Counting Bits

- We can iterate over the bits, then apply the mask to check each one.

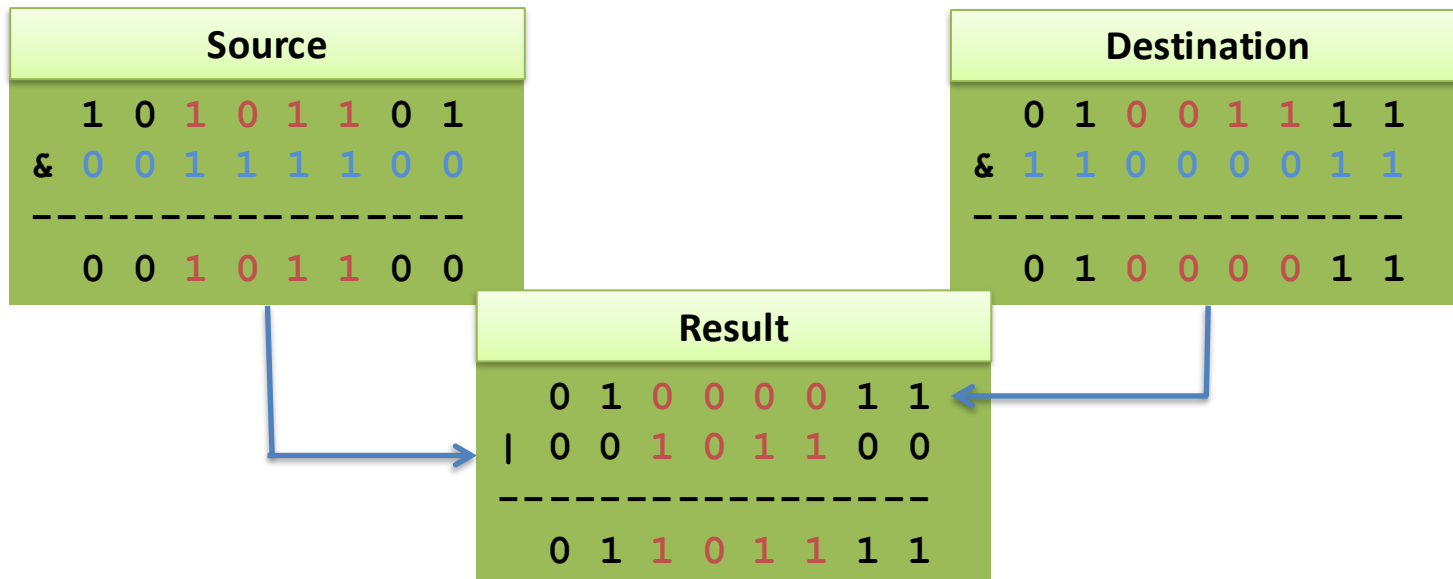
```
unsigned int secret = 2348291;
int bcount = 0;
for ( int i = 0; i < 8 * sizeof( int ); i++ ) {
    if ( secret & ( 1 << i ) )
        bcount++;
}
printf( "That's %d one bits\n", bcount );
```

Are these parentheses
needed?

- We could use the same technique to print a number in binary.

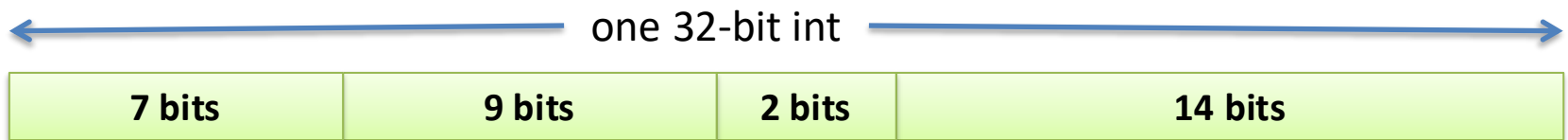
Copying Selected Bits

- Say we wanted to copy the middle four bits in a character
- In the source, we can use & to mask off everything but the bits we want to copy.
- In the destination, we can use & with a complementary mask to clear out these bits
- Then, we can use | to turn on just the bits copied from the source



Bit Fields

- We can pack multiple small integer values into a variable.
 - This can save some storage or reduce communication overhead.
 - It can even let us manipulate multiple values in parallel.



Extracting a Bit Field

- To extract a particular field:
 - Mask off the other bits
 - Then, shift down (so you can use the contents of the field like an int)
 - Say, we want to get just the middle 4 bits of a char.

1	0	1	0	1	1	0	1
&	0	0	1	1	1	1	0

0	0	1	0	1	1	0	0

0	0	1	0	1	1	0	0
>>						2	

				1 0 1 1			

Changing a Bit Field

- Then, we can manipulate the value of that field.
- And put the result back when we're done.

1	0	1	0	1	1	0	1
&	1	1	0	0	0	0	1

1	0	0	0	0	0	0	1

Original char,
make room for
the new field
value.

Insert the
updated value

1	0	0	0	0	0	0	1
	0	0	1	1	0	0	0

1	0	1	1	0	0	0	1

	1	0	1	1			
+				1			

	1	1	0	0			
<<				2			

	1	1	0	0	0	0	
&	0	0	1	1	1	0	0

0	0	1	1	0	0	0	0

Add one to
the field.

Shift it back
to where it
goes.

Mask off other
bits (in case of
overflow)